Assignment 3 Report: Representation Learning and Sequence Modeling (Online Retail)

Problem and data (what this study is about)

The Online Retail dataset (UCI / Kaggle) is invoice line items: each row is one product on an order (StockCode, quantities, prices, timestamps, CustomerID). The task is customer-level: find hidden groups of shoppers without using a hand-made label, then train neural sequence models that read a customer's purchase history and predict outcomes of interest.

Representation design and justification

Unit of analysis, each customer is one example. Their sequence is the time-ordered list of their purchase events (after cleaning invalid rows and restricting to customers used in modeling).

At each position in the sequence the model sees a categorical token for the product (StockCode) and numeric context for that line, Quantity, UnitPrice, and line amount, standardized using train-split means and standard deviations so scale does not dominate learning.

Sequences are truncated to a maximum length chosen from the data (about the 95th percentile of sequence length, capped at 120 events). Shorter sequences are padded; a mask tells the model which positions are real events vs padding.

Three customer-level tables for clustering. raw: aggregates (frequency, spend, recency, etc.). sequence: raw plus simple dynamics (e.g. spread of basket size, gaps between orders). latent: standardized sequence features → PCA (7 components, ≈0.95 explained variance) plus light K-means metadata (cluster id and distance to centroid) appended for that representation only.

Aggregates are stable summaries; sequence fields add cadence and variability; PCA removes redundancy before K-means. Supervised models use event sequences (tokens + line numerics); labels come from the latent k = 6 clustering below—not from reading the aggregate tables directly.

Unsupervised learning

K-means for k = 2 … 8 on each family. Silhouette (higher = tighter, well-separated clusters) drove k choice, with Calinski–Harabasz, Davies–Bouldin, and inertia as backup metrics.

On raw and sequence, the best silhouette often occurred at k = 2, but cluster sizes were extremely skewed: one very large "normal" cluster and a tiny second cluster. That pattern usually means "inliers vs outliers" rather than a rich set of customer personas. On latent, a choice of k = 6 gave a more balanced multi-cluster picture (still imbalanced, but not a single dominant-vs-dust split).

Class proportions used as supervised labels (from the latent k = 6 solution, encoded as classes 0–5): roughly 46% / 36% in the two largest groups, ≈8% each in the next two, ≈0.6% in the fifth, and ≈0.05% in the smallest, so downstream classification is strongly imbalanced.

Second method for sanity check, Ward agglomerative on the same latent matrix at k = 6. Compared to K-means labels: ARI = 0.942755, NMI = 0.884462; silhouette 0.380240 vs 0.368773—partitions match up to renaming clusters, so the six-way split is stable, not a K-means quirk. K-means stays primary for the k grid and speed; usual caveats: choose k, outliers, roughly spherical groups.

Supervised task, models, and training behavior

Each customer has a segment label from the latent k = 6 clustering. The models read the purchase sequence and try to predict that label. Intuitively: *if shopping patterns contain information about "customer type," a sequence model should recover the same groups the clustering found from summaries.*

Train / validation / test split. 4,338 customers were split without overlap (≈70% / 15% / 15% → 3,036 / 651 / 651) so no customer appears in more than one split. Product codes were mapped to integers using a vocabulary built from training customers only (about 3,600 tokens including special symbols) to avoid leakage.

Accuracy is the fraction of customers classified correctly. Under heavy imbalance it can be dominated by the largest classes, so we emphasize macro-F1: the unweighted average of F1 across classes, so rare segments count equally in the headline score.

A recurrent network with token embeddings, numeric projections, and a pooled hidden state was trained with cross-entropy. Training loss decreased, but validation loss and macro-F1 flattened early, which is classic overfitting; checkpoints favored macro-F1 on validation rather than raw accuracy. A class-weighted loss variant raised macro-F1 (about 0.25 → 0.29) at the cost of lower accuracy (about 0.69 → 0.57), showing an explicit bias–variance / objective tradeoff: weighting pushes the model away from always predicting the majority.

A small multi-head self-attention encoder with the same inputs and pooling readout was trained under the same protocol. It also overfit in loss (training loss kept improving while validation loss worsened after early epochs). Validation macro-F1 peaked around 0.33 then plateaued—better balanced performance than the unweighted GRU on this split, at the cost of more computation.

On two test customers, head-averaged weights used the last real line as query; top-weighted earlier positions were mapped to StockCodes and z-scored Quantity / UnitPrice / amount. Mass is non-uniform over time (see notebook bar and heatmap figures)—consistent with selective use of context, not a causal explanation of the predicted segment.

Comparative conclusions (GRU vs Transformer, test set)

Model	Accuracy	Macro-F1	Train time	Parameters
Transformer	0.5960	0.2946	≈9.65 s	292,566
GRU	0.6851	0.2498	≈6.65 s	288,326

The two models have similar parameter counts, so the gap is not mainly "more neurons" it is how sequence information is aggregated. Class-wise: the GRU gets zero F1 on several minority labels in this evaluation, while the Transformer recovers non-zero signal on some of those rare groups; both still struggle on the extremest tail classes where test support is tiny, so those F1s are noisy.

If the product goal is overall correctness or cheaper training, the GRU is preferable here. If the goal is fairer performance across segments (minority classes matter), macro-F1 favors the Transformer despite lower accuracy.

Limitations and takeaways

Clusters are a useful proxy, not ground truth: labels inherit imbalance and algorithmic bias. The latent k = 6 target is more actionable than a k = 2 “everyone vs outliers” split on raw/sequence space, and the hierarchical cross-check supports stability of that six-way structure. Next steps that would strengthen claims without changing the core story: rebalance or merge ultra-rare segments, and use repeated splits or cross-validation to quantify variance especially for tail classes where a handful of test rows can move metrics a lot.